

Épreuve E6 - Conception et développement d'applications (option SLAM)

Lien vers la documentation complète, précisant et décrivant, si cela n'a été fait au verso de la fiche, la réalisation

**ANNEXE 9-1-B : Fiche descriptive de réalisation professionnelle
(verso, éventuellement pages suivantes)**

Épreuve E6 - Conception et développement d'applications (option SLAM)

Descriptif de la réalisation professionnelle, y compris les productions réalisées et schémas explicatifs

1. Présentation.....3

2.Appercu visuel du projet.....3

3.Base de données.....6

4. Détails techniques.....5

 4.1 Connexion utilisateur.....7

 4.2 Liaison à la BDD supabase.....7

 4.3 Tableau de bord.....8

 4.4 Centralisation.....8

 4.5 Chargement des données.....8

 4.6 Requêtes CRUD (types).....9

 4.7 Requêtes CRUD (formats).....9

 4.8 Requêtes CRUD (stock).....9

ANNEXE 9-1-B : Fiche descriptive de réalisation professionnelle
(verso, éventuellement pages suivantes)

Épreuve E6 - Conception et développement d'applications (option SLAM)

I.Présentation

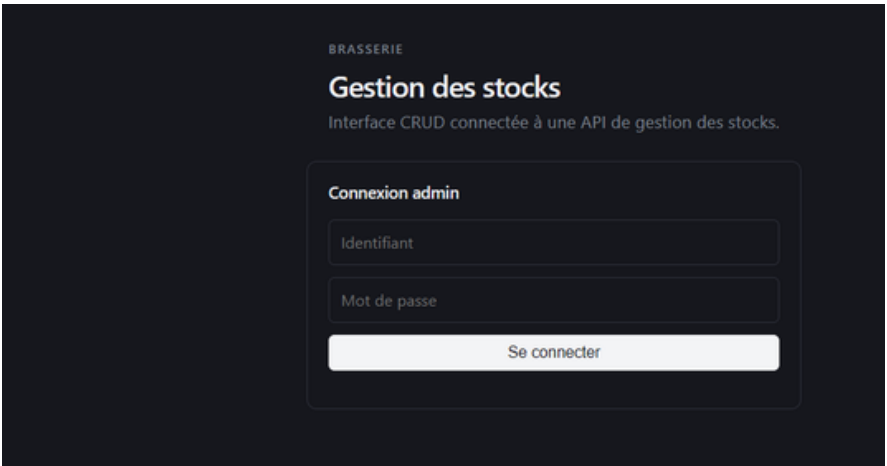
L'application permet la gestion du stock de produits via une interface web en React, connectée à une API REST (en Dart/Shelf).

L'objectif est de consulter et gérer les entités nécessaires au stockage :

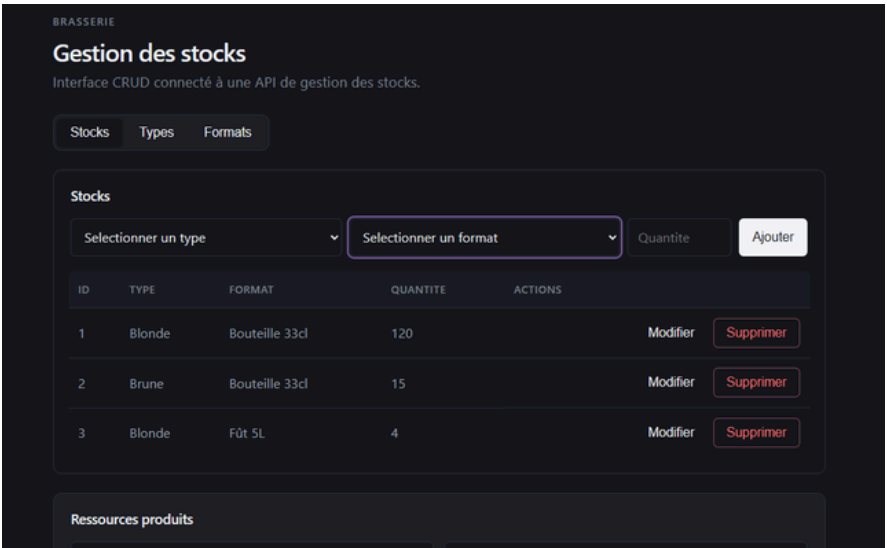
- les types de bière
- les formats
- les stocks



II.Apperçu du visuel



Voici, la page d'accueil, celle que vous voyez lorsque vous lancez l'application, il faut renseigner le login de l'administrateur. Uniquement lui peut se connecter.



Après avoir été login, l'admin arrive sur ce panneau de gestion des stocks, ou vous pouvez ajouter, modifier et supprimer du stock, également vous déconnecter.

Stocks

Selectionner un type

Selectionner un type

Blonde

Brune

Gin

IPA

Whisky

3 Blonde Fût 5L

Lorsque vous cliquez sur 'selectionner un type, plusieurs choix sont possible

Selectionner un format

Selectionner un format

Bouteille 33cl

Bouteille 75cl

Canette 50cl

Fût 5L

15

4

Même chose pour le format, vous pouvez choisir plusieurs contenances

Stocks

Selectionner un type

Selectionner un format

Quantite

Ajouter

ID	TYPE	FORMAT	QUANTITE	ACTIONS
1	Blonde	Bouteille 33cl	120	Modifier Supprimer
2	Brune	Bouteille 33cl	15	Modifier Supprimer

Ainsi, après avoir renseigné le type et le format, il suffit de renseigner la quantité et vous pouvez ajouter au stock la marchandise.

Il est impossible d'ajouter au stock tant que tout n'a pas été sélectionné.

	QUANTITE	ACTIONS
e 33cl	120	Modifier Supprimer
e 33cl	15	Modifier Supprimer
	4	Modifier Supprimer

Ici, vous pouvez modifier ou supprimer, le stock, le format ou la quantité d'une marchandise.

TypesFormatsStocks

Ajouter un stock

Type

Format

Quantité

Ajouter

Stocks



ID	TYPE	FORMAT	QUANTITÉ	ACTIONS
1			125	Modifier Supprimer
2			15	Modifier Supprimer
3			4	Modifier Supprimer
4			120	Modifier Supprimer
5			120	Modifier Supprimer

Ici, vous êtes sur la page types, tous les types sont renseignés, on peut les modifier ou les supprimer.

BRASSERIE

Déconnexion

Gestion des stocks

Interface CRUD connectée à une API de gestion des stocks.

StocksTypesFormats

Formats

Nouveau format (ex: Canette 33cl)

Ajouter

ID	LIBELLE	ACTIONS
1	Bouteille 33cl	Modifier Supprimer
2	Bouteille 75cl	Modifier Supprimer
3	Canette 50cl	Modifier Supprimer
4	Fût 5L	Modifier Supprimer

Ressources produits

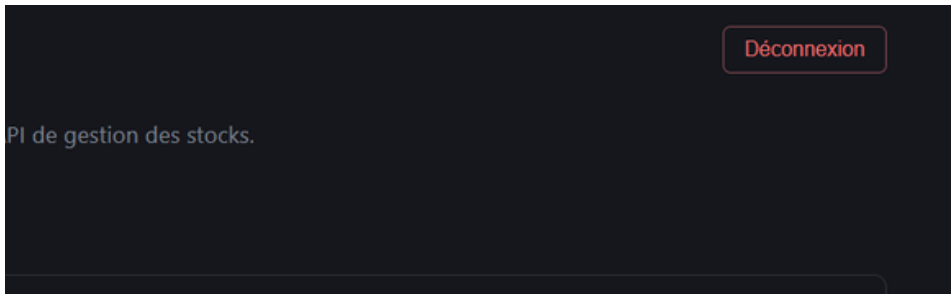
Biere Blonde

Biere Brune

Ici, vous êtes sur la page formats, tous les formats disponibles sont renseignés, on peut les ajouter, les supprimer.

ANNEXE 9-1-B : Fiche descriptive de réalisation professionnelle
(verso, éventuellement pages suivantes)

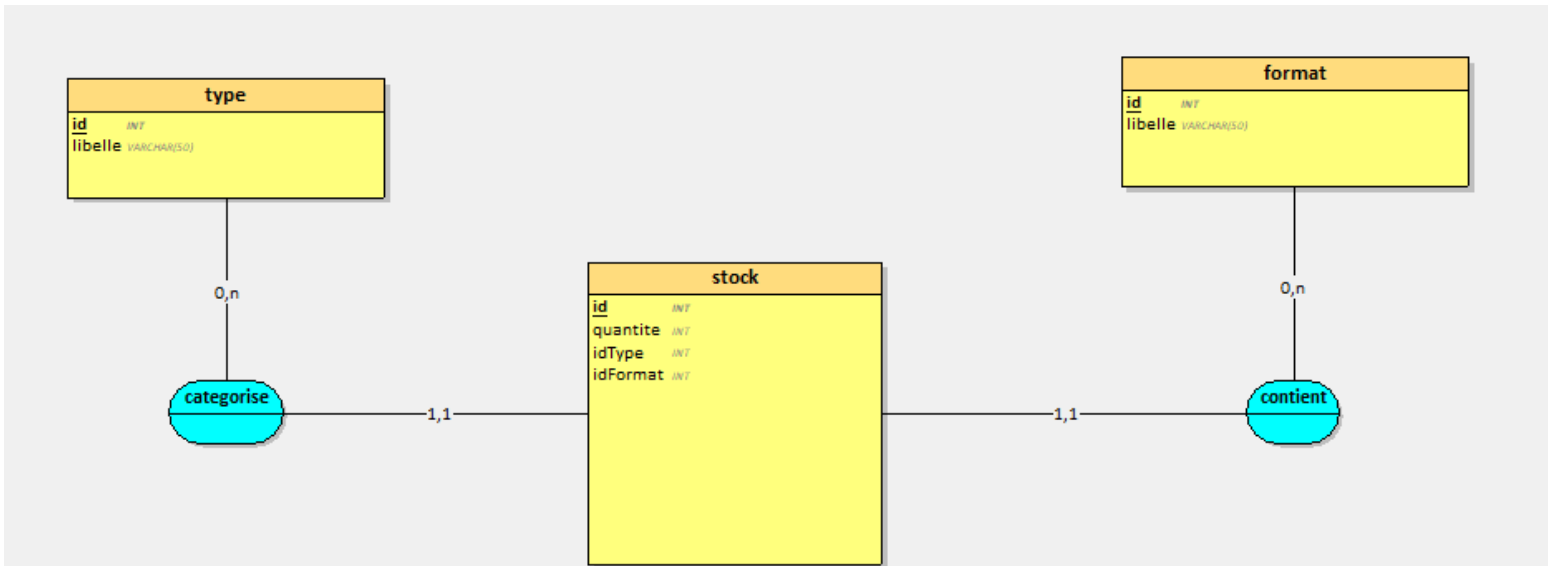
Épreuve E6 - Conception et développement d'applications (option SLAM)



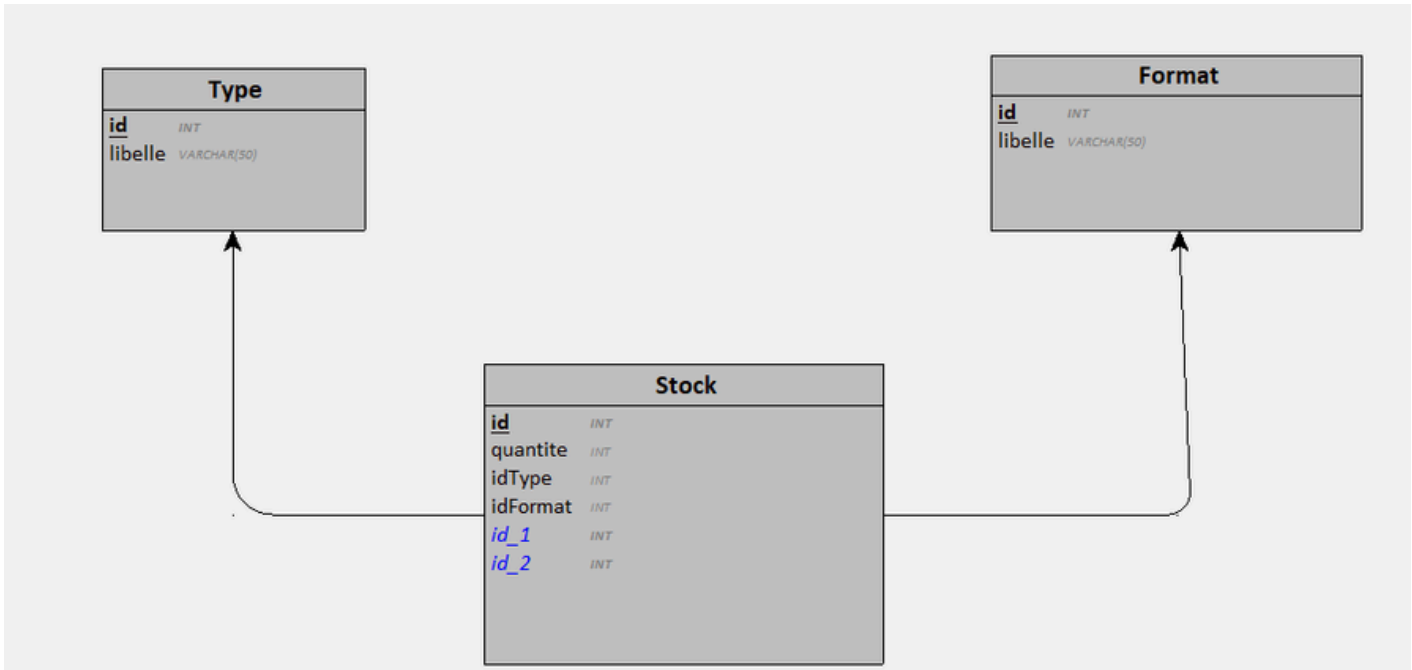
En haut à droite, un bouton déconnexion est placé pour pouvoir changer d'utilisateur.

III.Base de données

MCD de la BDD :



MCD de la BDD :



Épreuve E6 - Conception et développement d'applications (option SLAM)

admins	
<u>id</u>	INT
username	VARCHAR(50)
password	VARCHAR(50)
created_at	DATETIME

IV.Détails technique

```
function App() {
  const [isAuthenticated, setIsAuthenticated] = useState(false);
  const [activeTab, setActiveTab] = useState('stocks')

  return (
```

Le composant App() est le chef d'orchestre : il garde en mémoire si l'utilisateur est connecté, et selon ça il affiche soit le formulaire de connexion, soit le tableau de bord."

```
import { createClient } from '@supabase/supabase-js'

const supabaseUrl = 'https://hzfzqkjltnarsplbvubli.supabase.co'
const supabaseKey = 'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJ...

export const supabase = createClient(supabaseUrl, supabaseKey)
```

Ici, on fait la connexion à la base de données Supabase avec l'URL du projet et la anonKey.

ANNEXE 9-1-B : Fiche descriptive de réalisation professionnelle (verso, éventuellement pages suivantes)

Épreuve E6 - Conception et développement d'applications (option SLAM)

3 - Affichage du tableau de bord

```
const handleLogin = async (event) => {
  event.preventDefault();
  setLoginError('');
  setLoading(true);

  const { username, password } = loginForm;

  try {
    const { data: user, error } = await supabase
      .from('admins')
      .select('*')
      .eq('username', username.trim())
      .eq('password', password)
      .single();

    if (error || !user) {
      setLoginError('Identifiants invalides.');
```

On interroge la table admins dans Supabase sur le username et le password. Si aucun résultat, on affiche une erreur. Si ça correspond on fait basculer l'affichage dans App.jsx

Screen de AuthGestion.jsx

4 - Centralisation

```
async function api(path, method = 'GET', body = null) {
  const res = await fetch(path, {
    method,
    headers: body ? { 'Content-Type': 'application/json' } : {},
    body: body ? JSON.stringify(body) : null,
  });
  return res
}
```

Grâce a cette fonction on centralise les appels afin d'éviter de répéter le code à chaque requête.

Screen de StockGestion.jsx

5 - Chargements des données

```
const fetchAll = async () => {
  setLoading(true)
  try {
    const [t, f, s] = await Promise.all([
      fetch(`${API_URL}/types`).then(r => r.json()),
      fetch(`${API_URL}/formats`).then(r => r.json()),
      fetch(`${API_URL}/beers`).then(r => r.json()),
    ])
    setTypes(t); setFormats(f); setStocks(s)
  } catch {
    flash('Connexion API impossible.', true)
  } finally {
    setLoading(false)
  }
}
```

Ici les requêtes sont lancées en parallèles, d'abord les types, puis les formats, puis les stocks. Avec Promise.all, les 3 partent en même temps et on attend que toutes les 3 soient finies.

Screen de StockGestion.jsx

ANNEXE 9-1-B : Fiche descriptive de réalisation professionnelle (verso, éventuellement pages suivantes)

Épreuve E6 - Conception et développement d'applications (option SLAM)

6 - Requêtes CRUD (types)

```
const createType = async (e) => {
  e.preventDefault()
  const res = await api(`${API_URL}/types`, 'POST', { libelle: typeLabel.trim() })
  if (!res.ok) return flash('Échec création type.', true)
  setTypeLabel(''); flash('Type ajouté.');
```

Ici sont les requêtes CRUD pour les types, On utilise un window.confirm sur deleteType pour s'assurer de la décision prise par l'utilisateur

Screen de StockGestion.jsx

7 - Requêtes CRUD (format)

```
const createFormat = async (e) => {
  e.preventDefault()
  const res = await api(`${API_URL}/formats`, 'POST', { libelle: formatLabel.trim() })
  if (!res.ok) return flash('Échec création format.', true)
  setFormatLabel(''); flash('Format ajouté.');
```

Ici sont les requêtes CRUD pour les formats cette fois, Comme pour type on utilise un window.confirm sur deleteFormat pour s'assurer de la décision prise par l'utilisateur

Screen de StockGestion.jsx

8 - Requêtes CRUD (Stock)

```
const createStock = async (e) => {
  e.preventDefault()
  const res = await api(`${API_URL}/beers`, 'POST', toNumbers(stockForm))
  if (!res.ok) return flash('Échec création stock.', true)
  setStockForm(EMPTY_STOCK); flash('Stock ajouté.');
```

Ici sont les requêtes CRUD pour les stock, Comme pour type et format on utilise un window.confirm sur deleteStock pour s'assurer de la décision prise par l'utilisateur. De plus, on utilise la fonction saveStock pour sauvegarder les modifications.

Screen de StockGestion.jsx